

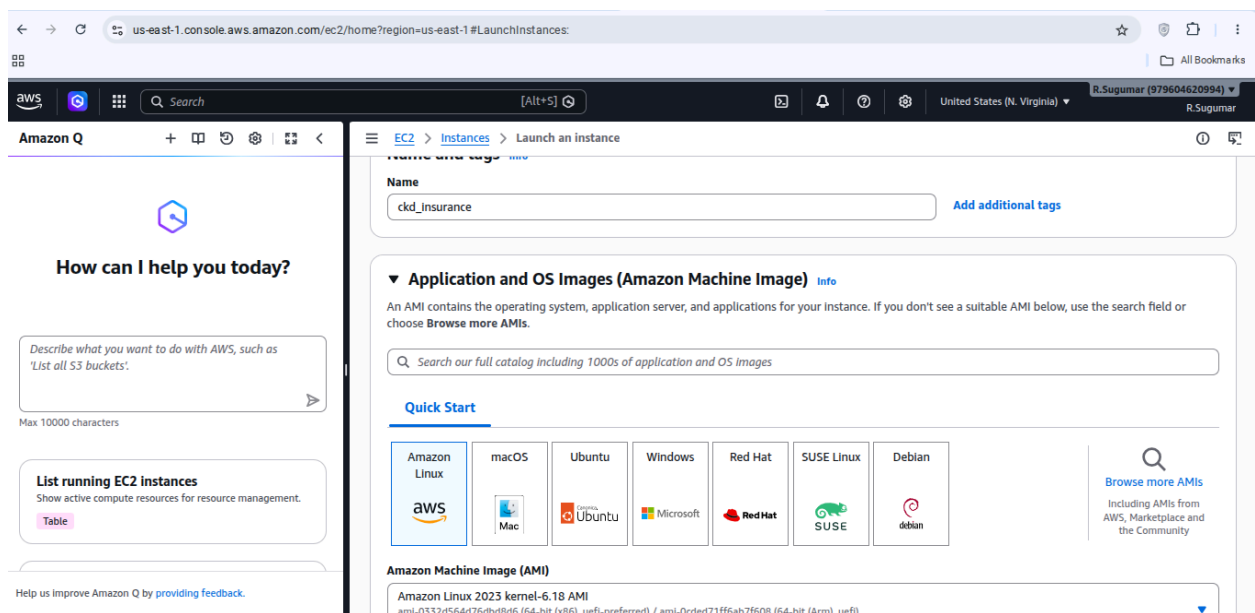
Amazon EC2 Dashboard in US East (N. Virginia).

Next step: Launch an EC2 instance

1. Click the orange **Launch instance** button in the middle of the page.
2. On the **Launch an instance** page, enter:
 - **Name:** `my-ec2-server`
 - **Application and OS Image (AMI):** Select **Amazon Linux 2023 AMI**
 - **Instance type:** `t2.micro` or `t3.micro` if available
 - **Key pair:** Select an existing key pair, or click **Create new key pair** if you don't have one.
3. Under **Network settings**:
 - Allow **SSH traffic** from **My IP**
 - Don't enable HTTP/HTTPS unless your project needs a web server.
4. Keep **Configure storage** at the default settings for now.
5. Click **Launch instance**.
6. After launching, click **View all instances**.
7. Wait until:
 - **Instance state:** `Running`
 - **Status check:** `2/2 checks passed`

For your next screenshot: after you click **Launch instance**, send me that page. I'll tell you exactly what to select in each field, one step at a time.

===



Yes 👍 You are in the correct place.

From your screenshot:

- **Name:** `ckd_insurance` ✓
- **AMI:** Amazon Linux 2023 ✓

Next step

Scroll down further.

You should find **Instance type**.

Select:

t2.micro — if it is available and marked Free Tier eligible.

Then continue scrolling until you see **Key pair (login)**.

⚠️ **Don't click Launch instance yet.**

Send me the next screenshot when you reach **Key pair (login)**, and I'll guide you from there.

====

The screenshot shows the AWS Management Console 'Launch an instance' page. The browser address bar shows 'us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#LaunchInstances:'. The page header includes the AWS logo, a search bar, and the user's profile 'R.Sugumar (979604620994)'. The main content area is divided into two sections: 'Amazon Q' on the left and 'Launch an instance' on the right. The 'Launch an instance' section is further divided into 'Amazon Machine Image (AMI)' and 'Instance type'. Under 'Amazon Machine Image (AMI)', the selected AMI is 'Amazon Linux 2023 kernel-6.18 AMI' with ID 'ami-0332d564d76dbd8d6'. The description states it is a modern, general purpose Linux-based OS supported until June 2029. Under 'Instance type', the selected type is 't3.micro', which is marked as 'Free tier eligible'. The page also shows the 'Key pair (login)' section, which is currently empty.

Perfect 👍 You have reached **Instance type**.

Your screenshot shows:

t3.micro — Free tier eligible ✓

Do this now

Keep **t3.micro** selected. You do **not** need to change it.

Then:

1. Scroll down.
2. Find **Key pair (login)**.
3. Look for the **Key pair name** dropdown.
4. If you already have a key pair, select it.
5. If you don't have one, **don't launch yet** — we'll create a new key pair.

After that, send me the screenshot of the **Key pair (login)** section.

====

4-bit x86 refers to the **CPU architecture** of the EC2 instance.

- **64-bit** → The processor can handle 64-bit operating systems and applications.
- **x86** → The common Intel/AMD processor architecture.
- **64-bit x86 (x86_64)** → Standard architecture used by most Intel and AMD servers.

In your screenshot, **64-bit (x86)** is the correct choice for your **Amazon Linux 2023 + t3.micro** instance. ✓

You can **leave it as 64-bit (x86)** and continue to **Key pair (login)**.

=====

▼ Instance type [Info](#) | [Get advice](#)

Instance type

t3.micro

Free tier eligible ▲

Family: t3 2 vCPU 1 GiB Memory Current generation: true On-Demand Linux base pricing: 0.0104 USD per Hour
On-Demand Windows base pricing: 0.0196 USD per Hour On-Demand Ubuntu Pro base pricing: 0.0139 USD per Hour
On-Demand SUSE base pricing: 0.0104 USD per Hour On-Demand RHEL base pricing: 0.0392 USD per Hour

es 👍 **t3.micro** is selected correctly.

Your screenshot shows:

- Instance type: **t3.micro** ✓
- 2 vCPU ✓
- 1 GiB memory ✓
- Free tier eligible ✓
- Current generation: Yes ✓

👉 Keep **t3.micro** as it is. Don't change anything here.

Next step

Scroll down until you see:

Key pair (login)

Then send me a screenshot of that section. I'll tell you exactly what to select/create.

====

▼ Key pair (login) [Info](#)

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name - *required*

Select ▼

🔄 Create new key pair

Perfect 👍 You are now at **Key pair (login)**.

Since the dropdown currently says **Select**, you need to create a key pair if you don't already have one.

Do this:

1. Click **Create new key pair**.
2. A new window will appear.
3. Enter:
 - **Key pair name:** `ckd-insurance-key`
 - **Key pair type:** `RSA`
 - **Private key file format:** `.pem`
4. Click **Create key pair**.

 Your browser will automatically download a file like:

`ckd-insurance-key.pem`

 **Very important:** Keep this `.pem` file safe. You need it later to connect to your EC2 server.

After the download finishes, **don't click Launch instance yet**. Send me a screenshot of the new key-pair window, and I'll check it for you.

===

Create key pair



Key pair name

Key pairs allow you to connect to your instance securely.

Enter key pair name

The name can include up to 255 ASCII characters. It can't include leading or trailing spaces.

Key pair type



RSA

RSA encrypted private and public key pair



ED25519

ED25519 encrypted private and public key pair

Private key file format



.pem

For use with OpenSSH



.ppk

For use with PuTTY



When prompted, store the private key in a secure and accessible location on your computer. **You will need it later to connect to your instance.** [Learn more](#)

Cancel

Create key pair

Good 👍 You have moved to **Network settings** → **Firewall (security groups)**.

Your current settings show:

- **Create security group** ✓
- **Allow SSH traffic** ✓
- **Source:** Anywhere (0.0.0.0/0) ⚠

Change this one setting

For **Allow SSH traffic from**, click the dropdown currently showing **Anywhere**.

Select:

My IP ✓

This is safer because SSH access will be limited to your current IP address.

Leave these unchecked

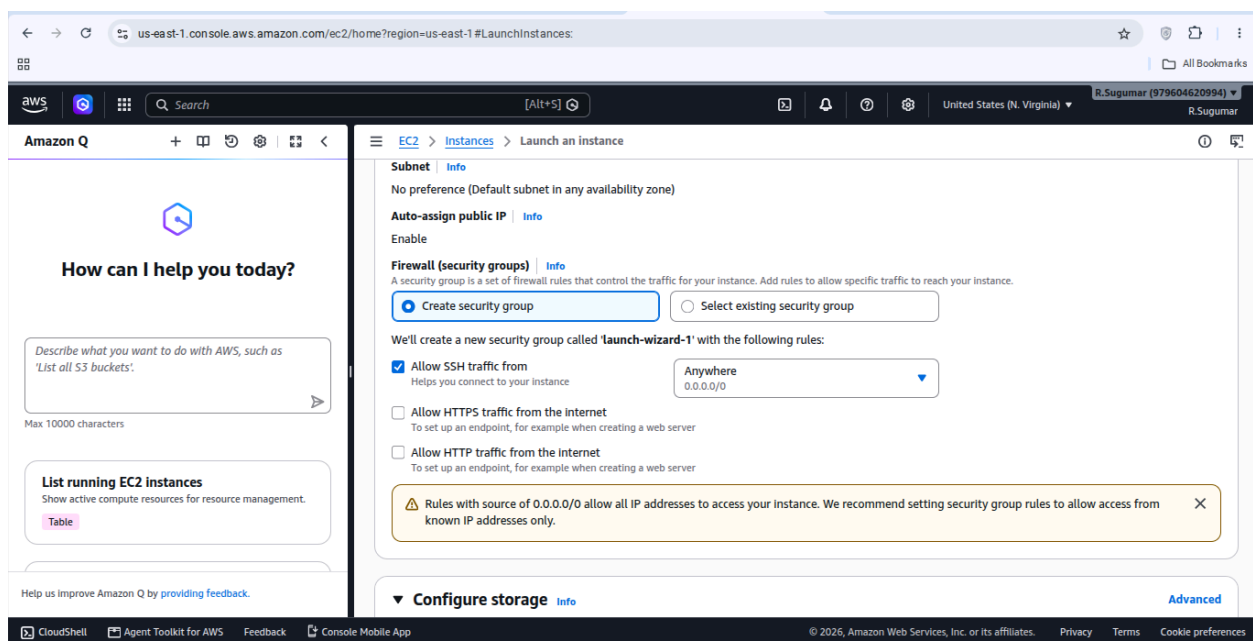
- Allow HTTPS traffic from the internet ❌
- Allow HTTP traffic from the internet ❌

You don't need them for a basic EC2 setup.

Then **scroll down to Configure storage**.

You can keep the default storage for now.

👉 **Don't click Launch instance yet.** Send me the next screenshot showing **Configure storage** and I'll guide you through it.



- ☒ Allow SSH traffic from Anywhere — selected
- ☒ Allow HTTPS traffic from the internet — selected
- ☒ Allow HTTP traffic from the internet — selected

So leave all three selected. ☒

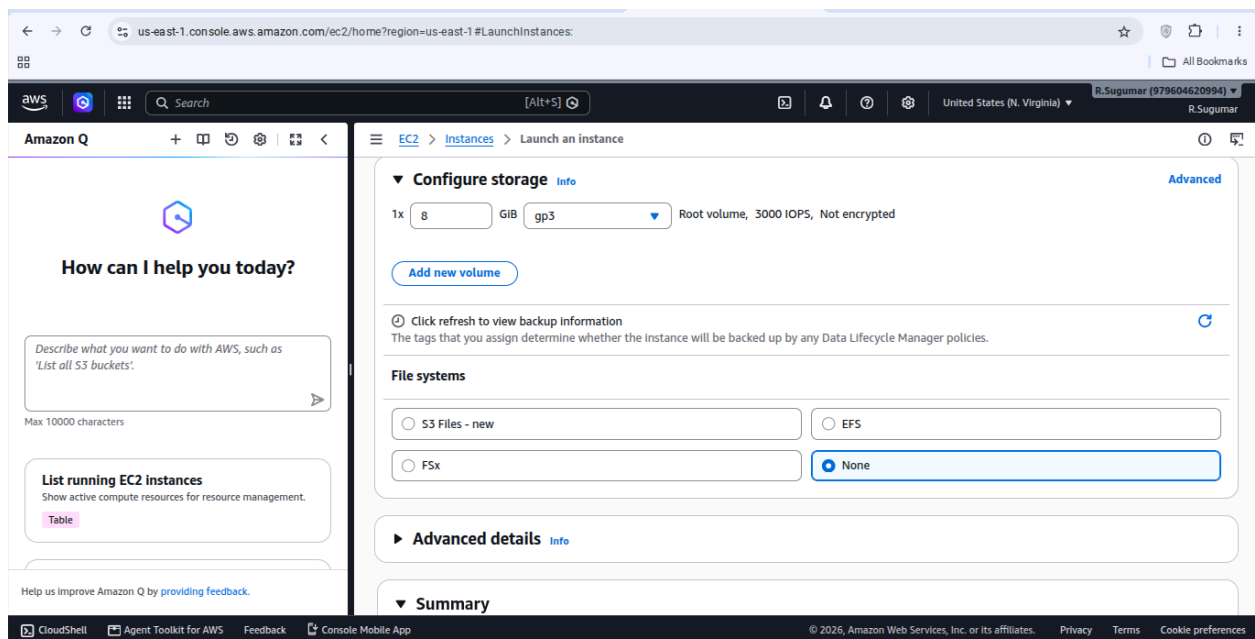
Next step

Now scroll down to **Configure storage**.

For storage, **don't change anything yet**. Keep the default settings unless your mam gave you a specific storage size.

Send me the screenshot of **Configure storage**, and I'll tell you what to select next.

===



es 👍 This **Configure storage** section is correct.

Keep the settings as they are:

- **Storage:** 8 GiB ✓
- **Volume type:** gp3 ✓
- **Root volume:** 1 volume ✓
- **File systems:** None ✓

You do not need to add a new volume.

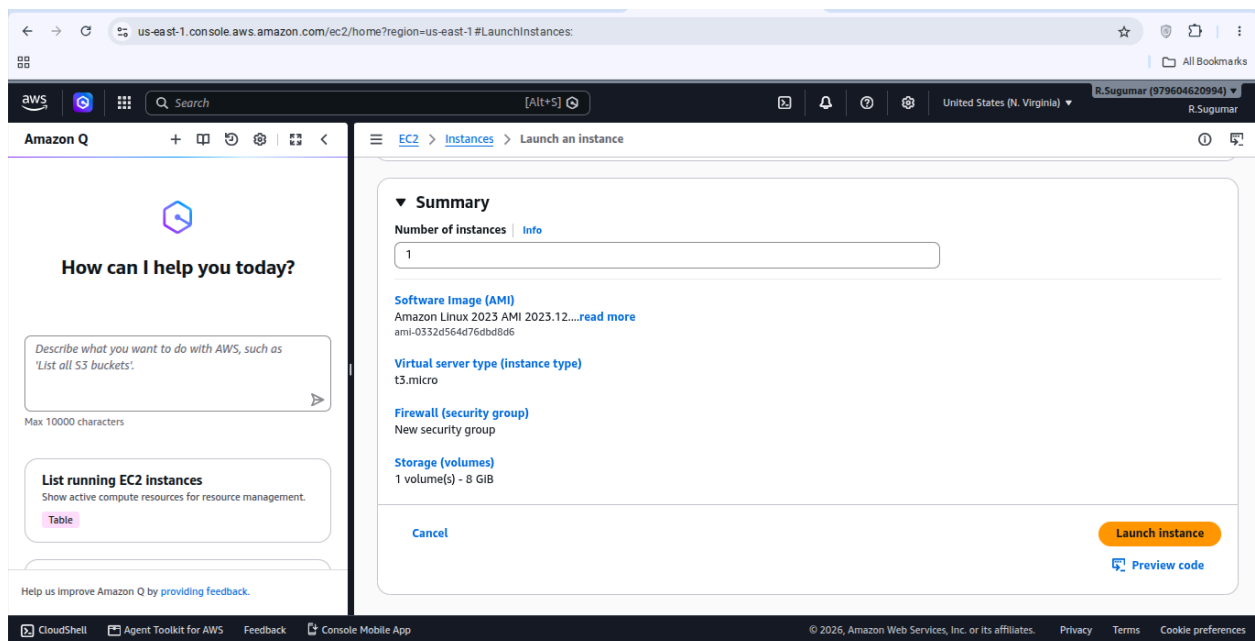
Next step

Now scroll a little further down to the **Summary** section.

You should see the orange **Launch instance** button.

Before clicking it, send me a screenshot of the **Summary** section. I'll check all the settings one final time.

=====



es 👍 Your **Summary** looks correct.

Final check

- **Number of instances:** 1 ✓
- **AMI:** Amazon Linux 2023 ✓
- **Instance type:** t3.micro ✓
- **Firewall:** New security group ✓
- **Storage:** 8 GiB ✓

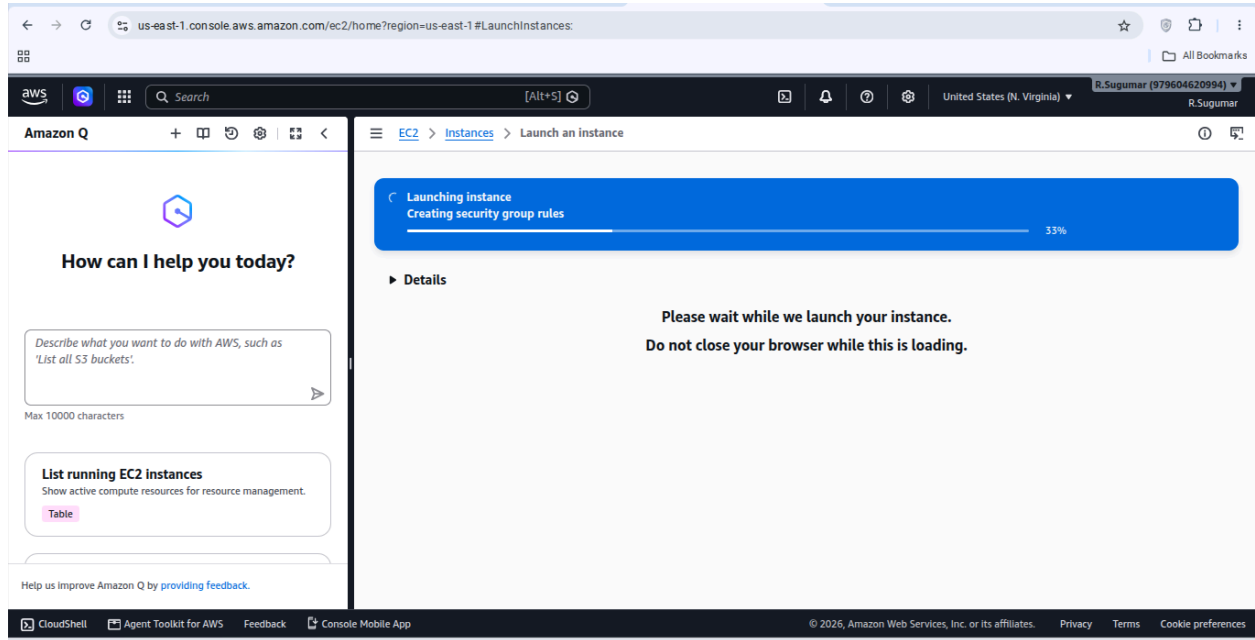
Everything looks ready.

👉 Now click the orange **Launch instance** button.

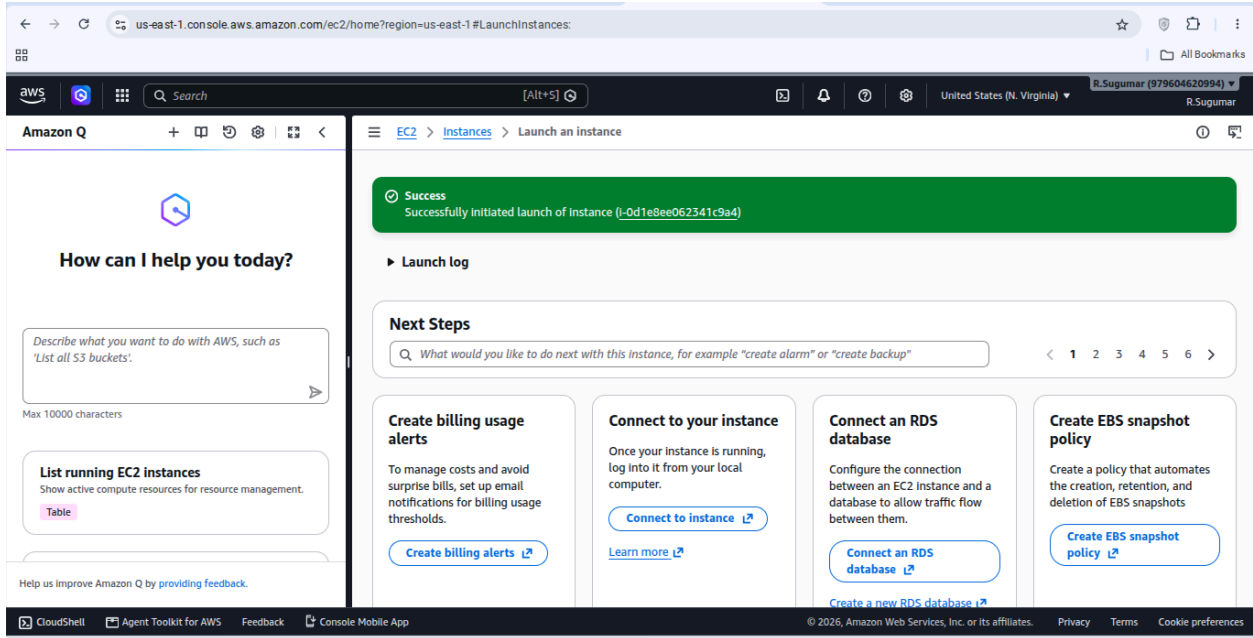
After you click it, AWS should show a message like **“Successfully initiated launch of instance”** and give you an **Instance ID**.

Send me the next screenshot after launching, and I'll guide you to the next step.

=====



=====



====

🎉 **Success! Your EC2 instance has been launched successfully.**

Your screenshot confirms:

Successfully initiated launch of instance

Next step

Click the **instance ID** shown in the green success message:

i-0d1e8ee062341c9a4

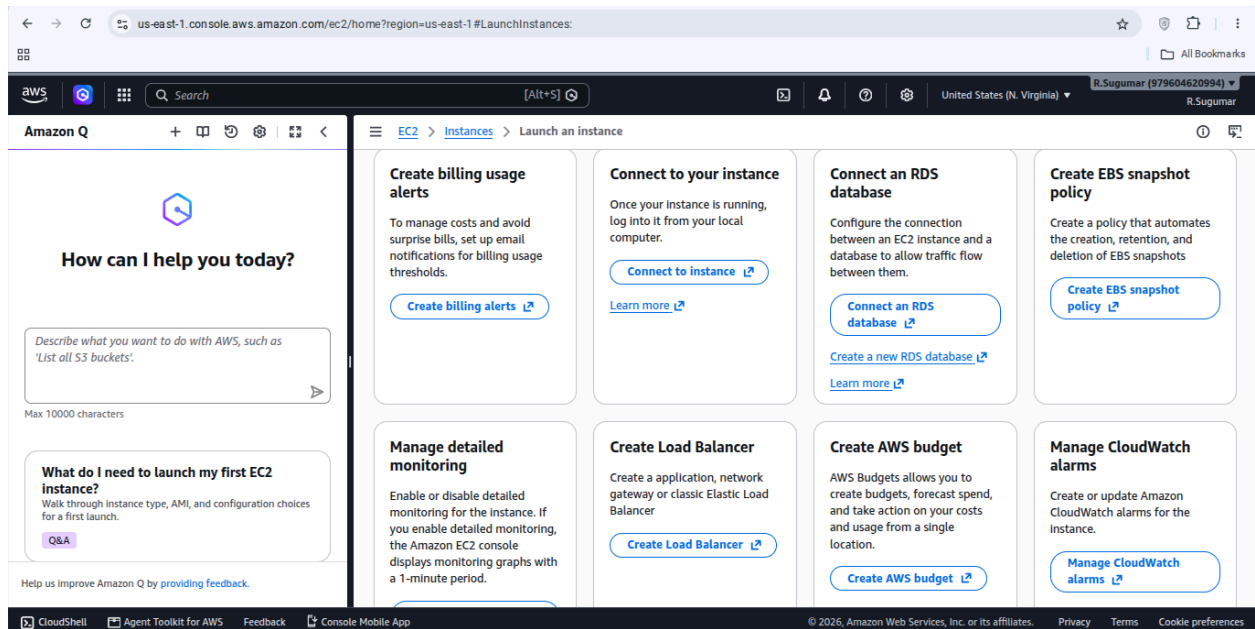
This will take you to the instance details page.

Then check:

- **Instance state:** Running ●
- **Status check:** 2/2 checks passed
- **Public IPv4 address:** should be displayed

👉 **Click the instance ID and send me the next screenshot.** Then I'll show you how to connect to this EC2 instance.

=====



Yes 👍 Your EC2 instance was successfully launched. This screenshot is showing the **Next Steps** section.

Next step: Connect to your EC2 instance

Click the blue **Connect to instance** button under **Connect to your instance**.

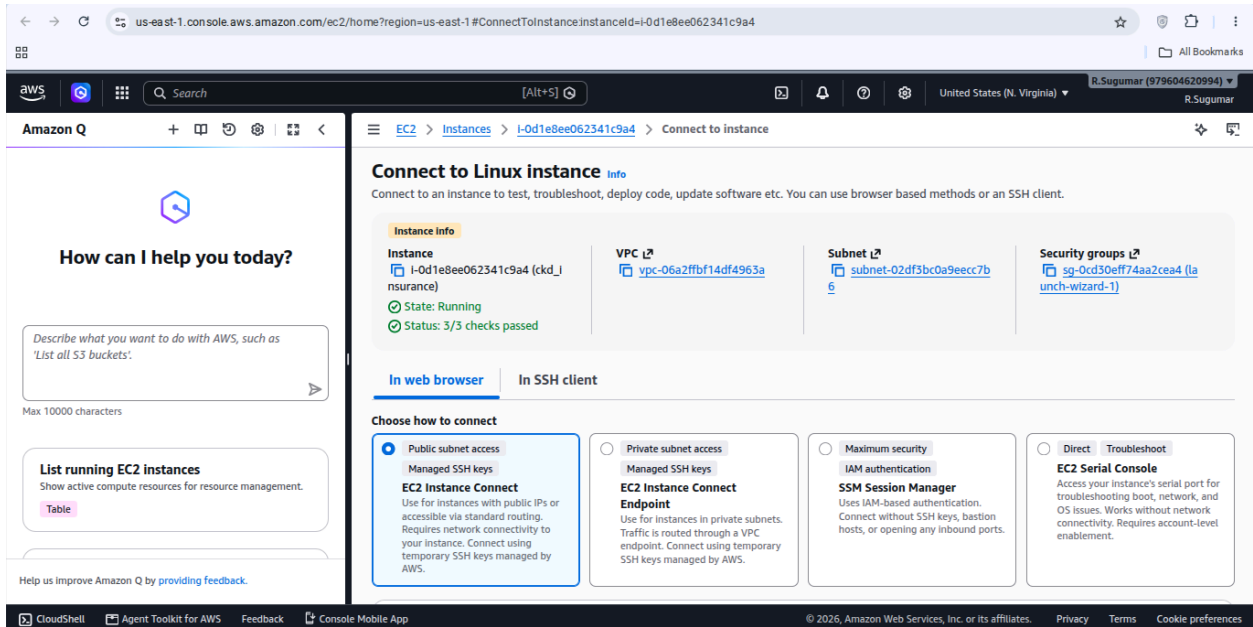
This will open the EC2 connection page.

On the next page, you should see options such as:

- **EC2 Instance Connect**
- **Session Manager**
- **SSH client**

👉 Click **Connect to instance now** and send me the next screenshot. I'll tell you exactly which connection option to choose.

=====



Excellent 👍 Your EC2 instance is **running successfully**.

Your screenshot shows:

- **State:** Running ✅
- **Status:** 3/3 checks passed ✅
- **Connection method:** **EC2 Instance Connect** is already selected ✅
- **Public subnet access:** selected ✅

Next step

Stay on **In web browser** and scroll down a little.

You should find an orange **Connect** button.

👉 Click **Connect**.

AWS should then open a **browser-based terminal** for your Linux instance.

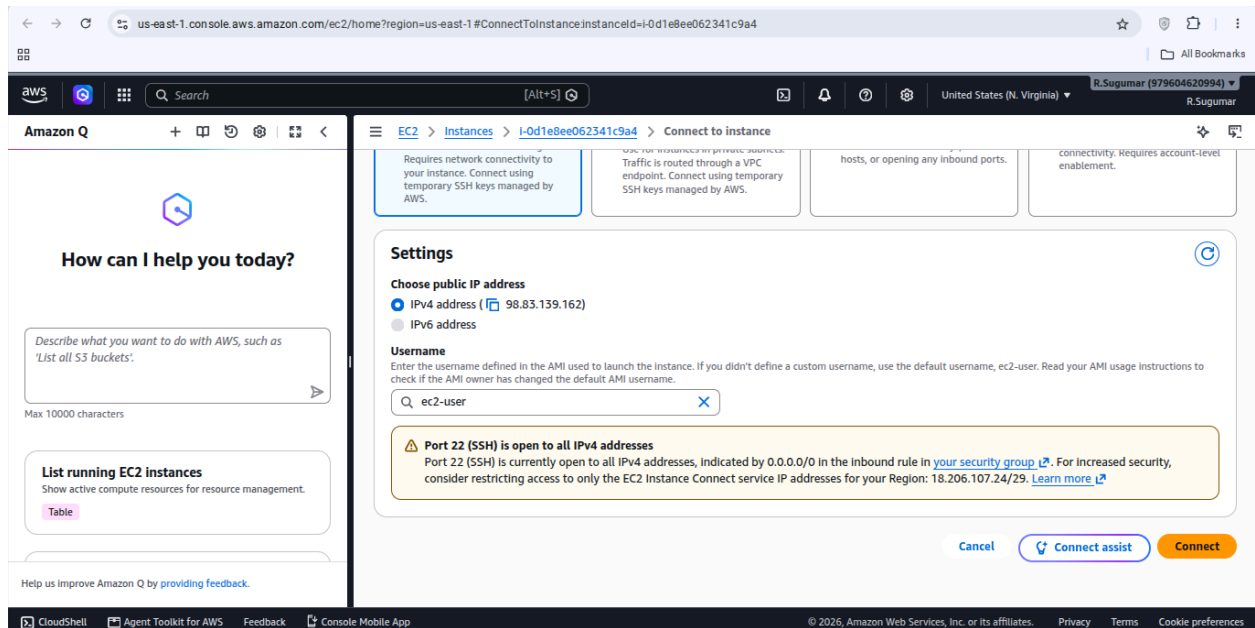
You should see something similar to:

```
[ec2-user@ip-xxx-xxx-xxx-xxx ~]$
```

That means you are successfully connected to your EC2 server.

Send me a screenshot after the terminal opens. Then I'll give you the next command to run.

====



Yes 👍 You are at the **final connection screen**.

Your settings are correct:

- **IPv4 address:** selected ✓
- **Public IP:** shown ✓
- **Username:** `ec2-user` ✓
- **Port 22 (SSH):** open ✓

Now click

🟡 **Connect**

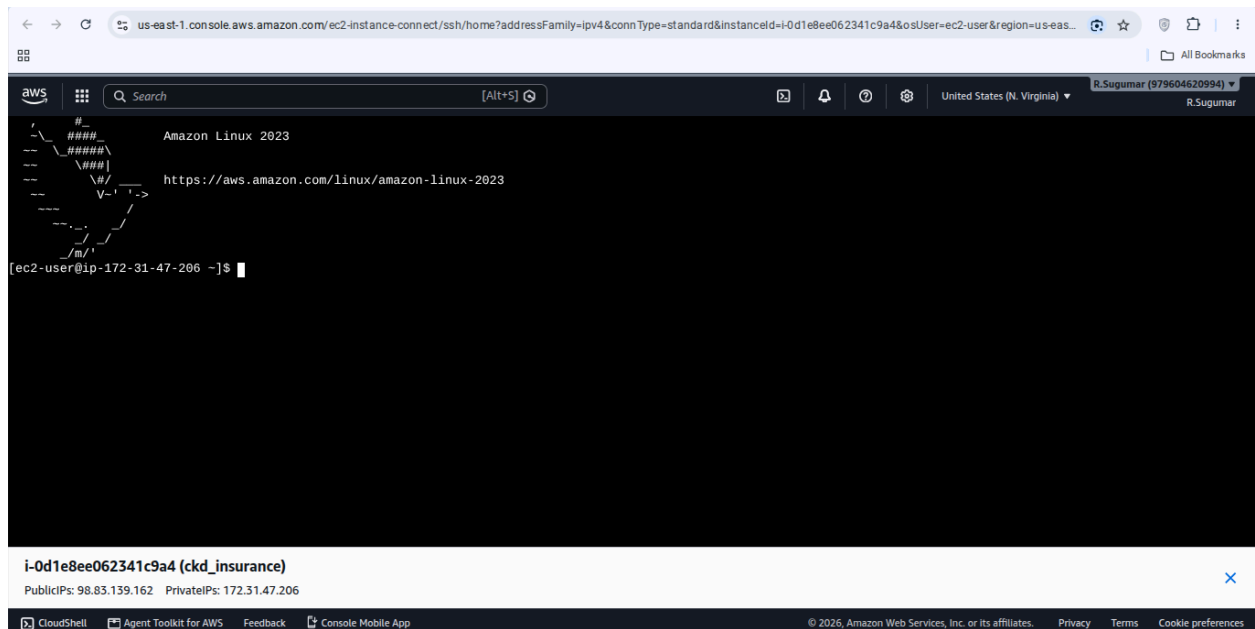
AWS should open the **EC2 Instance Connect terminal** in your browser.

Once the terminal opens, you should see a prompt similar to:

```
[ec2-user@ip-xxx-xxx-xxx-xxx ~]$
```

Send me a screenshot of that terminal. Then we'll proceed with installing and setting up whatever your project requires.

=====



🎉 **Excellent! You are successfully connected to your EC2 instance.**

The terminal shows:

```
[ec2-user@ip-172-31-47-206 ~]$
```

This means:

- **Amazon Linux 2023** is running ✓
- **EC2 instance is running** ✓
- **EC2 Instance Connect is working** ✓
- You now have a Linux command terminal on your AWS server ✓

Next step

Let's first verify the server with these commands.

Copy and paste **one command at a time** into the terminal:

whoami

Then:

python3 --version

Then:

pip3 --version

You should get outputs showing **ec2-user**, Python 3, and pip.

Send me the screenshot after running these commands. Then I'll guide you to the next setup step.

====

```
[ec2-user@ip-172-31-47-206 ~]$ sudo yum update -y
Amazon Linux 2023 Kernel Livepatch repository
Dependencies resolved.
Nothing to do.
Complete!
```

Perfect 👍 The command completed successfully.

Your output:

Dependencies resolved.
Nothing to do.
Complete!

means **Amazon Linux 2023 is already fully updated.** ✅

====


```
us-east-1.console.aws.amazon.com/ec2-instance-connect/ssh/home?addressFamily=ipv4&connType=standard&instanceId=i-0d1e8ee062341c9a4&osUser=ec2-user&region=us-east-1&s...
AWS Search [Alt+S] United States (N. Virginia) R.Sugumar (979604620994) R.Sugumar
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023
Last login: Tue Aug 25 05:42:22 2026 from 18.206.107.28
[ec2-user@ip-172-31-47-206 ~]$ sudo yum update -y
Amazon Linux 2023 Kernel Livepatch repository
Dependencies resolved.
Nothing to do.
Complete!
```

es 👍 I can see the terminal clearly.

Your previous command:

```
sudo yum update -y
```

finished successfully with:

Nothing to do.

Complete!

Now run the installation command

At the prompt, paste:

```
sudo yum install python3 python3-pip nginx -y
```

Press **Enter** and wait until it finishes.

You should eventually see:

Complete!

====

```
[ec2-user@ip-172-31-47-206 ~]$ sudo yum install python3 python3-pip nginx -y
```

Last metadata expiration check: 0:04:53 ago on Tue Aug 25 07:53:54 2026.

Package python3-3.9.25-1.amzn2023.0.9.x86_64 is already installed.

Dependencies resolved.

```
=====
=====
=====
```

Package	Architecture	Version
Repository	Size	

=====

=====

=====

Installing:

nginx	x86_64	1:1.30.4-1.amzn2023.0.1
amazonlinux	34 k	
python3-pip	noarch	21.3.1-2.amzn2023.0.20
amazonlinux	1.8 M	

Installing dependencies:

generic-logos-httpd	noarch	18.0.0-12.amzn2023.0.3
amazonlinux	19 k	
gperftools-libs	x86_64	2.9.1-1.amzn2023.0.3
amazonlinux	308 k	
libunwind	x86_64	1.4.0-5.amzn2023.0.3
amazonlinux	66 k	
nginx-core	x86_64	1:1.30.4-1.amzn2023.0.1
amazonlinux	711 k	
nginx-filesystem	noarch	1:1.30.4-1.amzn2023.0.1
amazonlinux	10 k	
nginx-mimetypes	noarch	2.1.49-3.amzn2023.0.3
amazonlinux	21 k	

Installing weak dependencies:

libxcrypt-compat	x86_64	4.4.33-7.amzn2023
amazonlinux	92 k	

Transaction Summary

=====

=====

=====

Install 9 Packages

Total download size: 3.0 M

Installed size: 15 M

Downloading Packages:

(1/9): generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch.rpm

485 kB/s | 19 kB 00:00

(2/9): libunwind-1.4.0-5.amzn2023.0.3.x86_64.rpm

1.5 MB/s | 66 kB 00:00

(3/9): gperftools-libs-2.9.1-1.amzn2023.0.3.x86_64.rpm

6.0 MB/s | 308 kB 00:00

(4/9): libxcrypt-compat-4.4.33-7.amzn2023.x86_64.rpm

4.4 MB/s | 92 kB 00:00

(5/9): nginx-1.30.4-1.amzn2023.0.1.x86_64.rpm

1.2 MB/s | 34 kB 00:00

(6/9): nginx-core-1.30.4-1.amzn2023.0.1.x86_64.rpm
22 MB/s | 711 kB 00:00
(7/9): nginxfilesystem-1.30.4-1.amzn2023.0.1.noarch.rpm
425 kB/s | 10 kB 00:00
(8/9): nginx-mimetypes-2.1.49-3.amzn2023.0.3.noarch.rpm
807 kB/s | 21 kB 00:00
(9/9): python3-pip-21.3.1-2.amzn2023.0.20.noarch.rpm
41 MB/s | 1.8 MB 00:00

Total 16 MB/s |
3.0 MB 00:00
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
Preparing :
1/1
Running scriptlet: nginxfilesystem-1:1.30.4-1.amzn2023.0.1.noarch
1/9
Installing : nginxfilesystem-1:1.30.4-1.amzn2023.0.1.noarch
1/9
Installing : nginx-mimetypes-2.1.49-3.amzn2023.0.3.noarch
2/9
Installing : libxcrypt-compat-4.4.33-7.amzn2023.x86_64
3/9
Installing : libunwind-1.4.0-5.amzn2023.0.3.x86_64
4/9
Installing : gperftools-libs-2.9.1-1.amzn2023.0.3.x86_64
5/9
Installing : nginx-core-1:1.30.4-1.amzn2023.0.1.x86_64
6/9
Installing : generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch
7/9
Installing : nginx-1:1.30.4-1.amzn2023.0.1.x86_64
8/9
Running scriptlet: nginx-1:1.30.4-1.amzn2023.0.1.x86_64
8/9
Installing : python3-pip-21.3.1-2.amzn2023.0.20.noarch
9/9
Running scriptlet: python3-pip-21.3.1-2.amzn2023.0.20.noarch
9/9

```
Verifying      : generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch
1/9
Verifying      : gperftools-libs-2.9.1-1.amzn2023.0.3.x86_64
2/9
Verifying      : libunwind-1.4.0-5.amzn2023.0.3.x86_64
3/9
Verifying      : libxcrypt-compat-4.4.33-7.amzn2023.x86_64
4/9
Verifying      : nginx-1:1.30.4-1.amzn2023.0.1.x86_64
5/9
Verifying      : nginx-core-1:1.30.4-1.amzn2023.0.1.x86_64
6/9
Verifying      : nginx-filesystem-1:1.30.4-1.amzn2023.0.1.noarch
7/9
Verifying      : nginx-mimetypes-2.1.49-3.amzn2023.0.3.noarch
8/9
Verifying      : python3-pip-21.3.1-2.amzn2023.0.20.noarch
9/9
```

Installed:

```
generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch
gperftools-libs-2.9.1-1.amzn2023.0.3.x86_64      libunwind-1.4.0-5.amzn2023.0.3.x86_64
libxcrypt-compat-4.4.33-7.amzn2023.x86_64
nginx-1:1.30.4-1.amzn2023.0.1.x86_64      nginx-core-1:1.30.4-1.amzn2023.0.1.x86_64
nginx-filesystem-1:1.30.4-1.amzn2023.0.1.noarch
nginx-mimetypes-2.1.49-3.amzn2023.0.3.noarch
python3-pip-21.3.1-2.amzn2023.0.20.noarch
```

Complete!

=====

It will install:

- **Python 3** → runs your SCI-BOT application
- **pip3** → installs Python packages from `requirements.txt`
- **Nginx** → can act as a web server/reverse proxy for your application

=====

```
[ec2-user@ip-172-31-47-206 ~]$ python3 --version
Python 3.9.25
[ec2-user@ip-172-31-47-206 ~]$ pip3 --version
pip 21.3.1 from /usr/lib/python3.9/site-packages/pip (python 3.9)
```

```
[ec2-user@ip-172-31-47-206 ~]$
```

====

Yes 👍 Run this in your EC2 terminal:

```
sudo pip3 install virtualenv
```

This installs **Virtualenv**, which lets you create an isolated Python environment for your project.

After it finishes, verify it with:

```
virtualenv --version
```

If you see a version number, it is installed successfully. ✅

Then we'll create the **virtual environment for your project**.

====

```
[ec2-user@ip-172-31-47-206 ~]$ sudo pip3 install virtualenv
```

```
Collecting virtualenv
```

```
  Downloading virtualenv-21.7.5-py3-none-any.whl (5.3 MB)
```

```
    |████████████████████████████████████████| 5.3 MB 9.4 MB/s
```

```
Collecting typing-extensions>=4.13.2
```

```
  Downloading typing_extensions-4.16.0-py3-none-any.whl (45 kB)
```

```
    |████████████████████████████████████████| 45 kB 1.7 MB/s
```

```
Collecting filelock<=3.19.1,>=3.16.1
```

```
  Downloading filelock-3.19.1-py3-none-any.whl (15 kB)
```

```
Collecting platformdirs<5,>=3.9.1
```

```
  Downloading platformdirs-4.4.0-py3-none-any.whl (18 kB)
```

```
Collecting python-discovery>=1.4.2
```

```
  Downloading python_discovery-1.5.3-py3-none-any.whl (38 kB)
```

```
Collecting distlib<1,>=0.3.7
```

```
  Downloading distlib-0.4.3-py2.py3-none-any.whl (470 kB)
```

```
    |████████████████████████████████████████| 470 kB 56.6 MB/s
```

```
Installing collected packages: filelock, typing-extensions, python-discovery, platformdirs, distlib, virtualenv
```

```
Successfully installed distlib-0.4.3 filelock-3.19.1 platformdirs-4.4.0 python-discovery-1.5.3 typing-extensions-4.16.0 virtualenv-21.7.5
```

WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager. It is recommended to use a virtual environment instead: <https://pip.pypa.io/warnings/venv>

====

Perfect 👍 **Virtualenv installed successfully.**

The important line is:

Successfully installed ... virtualenv-21.7.5

The warning is normal because you used `sudo`; it is **not an error**.

====

effect 👍 Both are installed correctly.

- **Python:** 3.9.25 ✓
- **pip:** 21.3.1 ✓

Now let's create the virtual environment for your **CKD Insurance project**.

Run these commands **one at a time**:

```
mkdir ckd_insurance
cd ckd_insurance
virtualenv venv
```

Then activate it:

```
source venv/bin/activate
```

After activation, your prompt should change to something like:

```
(venv) [ec2-user@ip-172-31-47-206 ckd_insurance]$
```

Send me the screenshot after activation. Then we'll continue with the next deployment step.

====

```
[ec2-user@ip-172-31-47-206 ~]$ mkdir ckd_insurance
```

```
[ec2-user@ip-172-31-47-206 ~]$ cd ckd_insurance
[ec2-user@ip-172-31-47-206 ckd_insurance]$ python3 -m venv venv
[ec2-user@ip-172-31-47-206 ckd_insurance]$ source venv/bin/activate
(venv) [ec2-user@ip-172-31-47-206 ckd_insurance]$
```

====

Excellent!  Your virtual environment is created and activated successfully.

Your prompt now shows:

```
(venv) [ec2-user@ip-172-31-47-206 ckd_insurance]$
```

That confirms you are working inside the **CKD Insurance project environment**.

=====

es  Since your virtual environment is active:

```
(venv) [ec2-user@ip-172-31-47-206 ckd_insurance]$
```

run:

```
pip install flask gunicorn
```

This installs:

- **Flask** → runs your web application
- **Gunicorn** → production WSGI server used to run Flask on EC2

After installation finishes, verify:

```
pip show flask
```

and:

```
pip show gunicorn
```

If both show package information, installation is successful. 

Don't use `sudo` here because you are already inside the virtual environment.

====

```
(venv) [ec2-user@ip-172-31-47-206 ckd_insurance]$ pip show flask
Name: Flask
Version: 3.1.3
Summary: A simple framework for building complex web applications.
Home-page:
Author:
Author-email:
License:
Location: /home/ec2-user/ckd_insurance/venv/lib/python3.9/site-packages
Requires: blinker, click, importlib-metadata, itsdangerous, jinja2, markupsafe, werkzeug
Required-by:
(venv) [ec2-user@ip-172-31-47-206 ckd_insurance]$ pip show gunicorn
Name: gunicorn
Version: 23.0.0
Summary: WSGI HTTP Server for UNIX
Home-page:
Author:
Author-email: Benoit Chesneau <benoitc@gunicorn.org>
License: MIT
Location: /home/ec2-user/ckd_insurance/venv/lib/python3.9/site-packages
Requires: packaging
Required-by:
(venv) [ec2-user@ip-172-31-47-206 ckd_insurance]$
```

=====

Perfect 👍 Both packages are installed correctly inside your virtual environment.

- **Flask:** 3.1.3 ✓
- **Gunicorn:** 23.0.0 ✓
- Installation location is inside your `ckd_insurance/venv` ✓
- No `sudo` was used ✓

====

es 👍 Run this command:


```
sudo yum install git -y
```

This installs **Git** on your EC2 server so you can download your CKD Insurance project from GitHub.

After it finishes, verify:

```
git --version
```

You should see something like:

```
git version 2.x.x
```

Then we can **clone your project repository into the `ckd_insurance` folder.**

====

```
(venv) [ec2-user@ip-172-31-47-206 ckd_insurance]$ pip show flask
```

Name: Flask

Version: 3.1.3

Summary: A simple framework for building complex web applications.

Home-page:

Author:

Author-email:

License:

Location: /home/ec2-user/ckd_insurance/venv/lib/python3.9/site-packages

Requires: blinker, click, importlib-metadata, itsdangerous, jinja2, markupsafe, werkzeug

Required-by:

```
(venv) [ec2-user@ip-172-31-47-206 ckd_insurance]$ pip show gunicorn
```

Name: gunicorn

Version: 23.0.0

Summary: WSGI HTTP Server for UNIX

Home-page:

Author:

Author-email: Benoit Chesneau <benoitc@gunicorn.org>

License: MIT

Location: /home/ec2-user/ckd_insurance/venv/lib/python3.9/site-packages

Requires: packaging

Required-by:

```
(venv) [ec2-user@ip-172-31-47-206 ckd_insurance]$ sudo yum install git -y
```

Last metadata expiration check: 0:19:54 ago on Tue Aug 25 07:53:54 2026.

Dependencies resolved.

=====		
=====		
=====		
Package	Architecture	Version
Repository	Size	
=====		
=====		
=====		

Installing:

git	x86_64	2.50.1-1.amzn2023.0.1
amazonlinux	53 k	
Installing dependencies:		
git-core	x86_64	2.50.1-1.amzn2023.0.1
amazonlinux	4.9 M	
git-core-doc	noarch	2.50.1-1.amzn2023.0.1
amazonlinux	2.8 M	
perl-Error	noarch	1:0.17030-2.amzn2023.0.1
amazonlinux	42 k	
perl-File-Find	noarch	1.37-477.amzn2023.0.9
amazonlinux	26 k	
perl-Git	noarch	2.50.1-1.amzn2023.0.1
amazonlinux	41 k	
perl-TermReadKey	x86_64	2.38-9.amzn2023.0.3
amazonlinux	36 k	
perl-lib	x86_64	0.65-477.amzn2023.0.9
amazonlinux	15 k	

Transaction Summary

=====		
=====		
=====		

Install 8 Packages

Total download size: 7.9 M

Installed size: 41 M

Downloading Packages:

- (1/8): git-2.50.1-1.amzn2023.0.1.x86_64.rpm
1.3 MB/s | 53 kB 00:00
- (2/8): git-core-doc-2.50.1-1.amzn2023.0.1.noarch.rpm
47 MB/s | 2.8 MB 00:00
- (3/8): perl-Error-0.17030-2.amzn2023.0.1.noarch.rpm
1.6 MB/s | 42 kB 00:00
- (4/8): git-core-2.50.1-1.amzn2023.0.1.x86_64.rpm
53 MB/s | 4.9 MB 00:00

(5/8): perl-File-Find-1.37-477.amzn2023.0.9.noarch.rpm
771 kB/s | 26 kB 00:00
(6/8): perl-Git-2.50.1-1.amzn2023.0.1.noarch.rpm
1.3 MB/s | 41 kB 00:00
(7/8): perl-TermReadKey-2.38-9.amzn2023.0.3.x86_64.rpm
1.3 MB/s | 36 kB 00:00
(8/8): perl-lib-0.65-477.amzn2023.0.9.x86_64.rpm
547 kB/s | 15 kB 00:00

Total 47 MB/s |
7.9 MB 00:00
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
Preparing :
1/1
Installing : git-core-2.50.1-1.amzn2023.0.1.x86_64
1/8
Installing : git-core-doc-2.50.1-1.amzn2023.0.1.noarch
2/8
Installing : perl-lib-0.65-477.amzn2023.0.9.x86_64
3/8
Installing : perl-TermReadKey-2.38-9.amzn2023.0.3.x86_64
4/8
Installing : perl-File-Find-1.37-477.amzn2023.0.9.noarch
5/8
Installing : perl-Error-1:0.17030-2.amzn2023.0.1.noarch
6/8
Installing : perl-Git-2.50.1-1.amzn2023.0.1.noarch
7/8
Installing : git-2.50.1-1.amzn2023.0.1.x86_64
8/8
Running scriptlet: git-2.50.1-1.amzn2023.0.1.x86_64
8/8
Verifying : git-2.50.1-1.amzn2023.0.1.x86_64
1/8
Verifying : git-core-2.50.1-1.amzn2023.0.1.x86_64
2/8
Verifying : git-core-doc-2.50.1-1.amzn2023.0.1.noarch
3/8

```
Verifying      : perl-Error-1:0.17030-2.amzn2023.0.1.noarch
4/8
Verifying      : perl-File-Find-1.37-477.amzn2023.0.9.noarch
5/8
Verifying      : perl-Git-2.50.1-1.amzn2023.0.1.noarch
6/8
Verifying      : perl-TermReadKey-2.38-9.amzn2023.0.3.x86_64
7/8
Verifying      : perl-lib-0.65-477.amzn2023.0.9.x86_64
8/8
```

Installed:

```
git-2.50.1-1.amzn2023.0.1.x86_64      git-core-2.50.1-1.amzn2023.0.1.x86_64
git-core-doc-2.50.1-1.amzn2023.0.1.noarch
perl-Error-1:0.17030-2.amzn2023.0.1.noarch
perl-File-Find-1.37-477.amzn2023.0.9.noarch    perl-Git-2.50.1-1.amzn2023.0.1.noarch
perl-TermReadKey-2.38-9.amzn2023.0.3.x86_64
perl-lib-0.65-477.amzn2023.0.9.x86_64
```

Complete!

(venv) [ec2-user@ip-172-31-47-206 ckd_insurance]\$

=====

Run this exact command:

```
git clone https://github.com/sugumarranganathan/awslocalmodelhosting.git
```

You should then see messages similar to:

```
Cloning into 'awslocalmodelhosting'...
remote: Enumerating objects...
Receiving objects...
Resolving deltas...
```

After it finishes, run:

ls

You should see:

```
awslocalmodelhosting
venv
```

=====

```
(venv) [ec2-user@ip-172-31-47-206 ckd_insurance]$ ls  
awslocalmodelhosting venv
```

=====

Excellent! 🎉 **The Git clone was successful.**

The important line is:

Resolving deltas: 100% (4/4), done.

Your project is now downloaded to:

ckd_insurance/awslocalmodelhosting

Next step

Run:

```
cd awslocalmodelhosting
```

Then:

```
ls -la
```

You should see files such as:

```
app.py  
requirements.txt  
finalized_model_ckd.sav  
templates/
```

Send me the `ls -la` output. Then we'll install the requirements for this exact project.

=====

```
(venv) [ec2-user@ip-172-31-47-206 ckd_insurance]$ git clone  
https://github.com/sugumarranganathan/awslocalmodelhosting.git  
Cloning into 'awslocalmodelhosting'...  
remote: Enumerating objects: 24, done.  
remote: Counting objects: 100% (24/24), done.
```

```
remote: Compressing objects: 100% (21/21), done.
remote: Total 24 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (24/24), 12.39 KiB | 12.39 MiB/s, done.
Resolving deltas: 100% (4/4), done.
(venv) [ec2-user@ip-172-31-47-206 ckd_insurance]$
```

===

Excellent! 🎉 **The Git clone was successful.**

The important line is:

Resolving deltas: 100% (4/4), done.

Your project is now downloaded to:

ckd_insurance/awslocalmodelhosting

Next step

Run:

```
cd awslocalmodelhosting
```

Then:

```
ls -la
```

You should see files such as:

```
app.py
requirements.txt
finalized_model_ckd.sav
templates/
```

Send me the `ls -la` output. Then we'll install the requirements for this exact project.

====

```
(venv) [ec2-user@ip-172-31-47-206 ckd_insurance]$ cd awslocalmodelhosting
(venv) [ec2-user@ip-172-31-47-206 awslocalmodelhosting]$ ls -la
total 20
drwxr-xr-x. 5 ec2-user ec2-user 166 Aug 25 08:36 .
drwxr-xr-x. 4 ec2-user ec2-user  46 Aug 25 08:36 ..
drwxr-xr-x. 7 ec2-user ec2-user 147 Aug 25 08:36 .git
```

```
drwxr-xr-x. 2 ec2-user ec2-user 102 Aug 25 08:36 __pycache__
-rw-r--r--. 1 ec2-user ec2-user 1384 Aug 25 08:36 app.py
-rw-r--r--. 1 ec2-user ec2-user 4055 Aug 25 08:36 fetching.ipynb
-rw-r--r--. 1 ec2-user ec2-user 3275 Aug 25 08:36 finalized_model_ckd.sav
-rw-r--r--. 1 ec2-user ec2-user 49 Aug 25 08:36 libraries.txt
-rw-r--r--. 1 ec2-user ec2-user 93 Aug 25 08:36 requirements.txt
drwxr-xr-x. 2 ec2-user ec2-user 59 Aug 25 08:36 templates
```

=====

```
[ec2-user@ip-172-31-47-206 ckd_insurance]$ cd awslocalmodelhosting
[ec2-user@ip-172-31-47-206 awslocalmodelhosting]$ source venv/bin/activate
-bash: venv/bin/activate: No such file or directory
[ec2-user@ip-172-31-47-206 awslocalmodelhosting]$ python3 -m venv venv
[ec2-user@ip-172-31-47-206 awslocalmodelhosting]$ source venv/bin/activate
(venv) [ec2-user@ip-172-31-47-206 awslocalmodelhosting]$ ls -la
total 20
drwxr-xr-x. 6 ec2-user ec2-user 178 Aug 25 12:30 .
drwxr-xr-x. 4 ec2-user ec2-user 46 Aug 25 08:36 ..
drwxr-xr-x. 7 ec2-user ec2-user 147 Aug 25 08:36 .git
drwxr-xr-x. 2 ec2-user ec2-user 102 Aug 25 08:36 __pycache__
-rw-r--r--. 1 ec2-user ec2-user 1384 Aug 25 08:36 app.py
-rw-r--r--. 1 ec2-user ec2-user 4055 Aug 25 08:36 fetching.ipynb
-rw-r--r--. 1 ec2-user ec2-user 3275 Aug 25 08:36 finalized_model_ckd.sav
-rw-r--r--. 1 ec2-user ec2-user 49 Aug 25 08:36 libraries.txt
-rw-r--r--. 1 ec2-user ec2-user 93 Aug 25 08:36 requirements.txt
drwxr-xr-x. 2 ec2-user ec2-user 59 Aug 25 08:36 templates
drwxr-xr-x. 5 ec2-user ec2-user 74 Aug 25 12:30 venv
(venv) [ec2-user@ip-172-31-47-206 awslocalmodelhosting]$
```

=====

Perfect 👍 Everything is correct now.

You successfully:

- Entered your cloned GitHub project: `awslocalmodelhosting` ✓
- Created a new virtual environment inside the project ✓

- Activated it successfully: (venv) ✓
- Confirmed all project files are present ✓

Your project contains:

app.py
requirements.txt
finalized_model_ckd.sav
templates/
fetching.ipynb
libraries.txt

Next step — install project dependencies

Since your virtual environment is active, run:

```
pip install -r requirements.txt
```

Do not use `sudo`.

Wait for the installation to finish. If you see:

Successfully installed ...

that's good.

Then **send me the output/screenshot**. We'll next run your `app.py` and test the CKD Insurance application.

=====

```
numpy==1.26.4  
pandas==2.2.2  
wheel==0.43.0  
scikit-learn==1.3.2  
matplotlib==3.8.4  
flask==3.0.3
```

=====

```
(venv) [ec2-user@ip-172-31-47-206 awslocalmodelhosting]$ pip install -r requirements.txt  
Collecting numpy==1.26.4
```


Downloading numpy-1.26.4-cp39-cp39-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (18.2 MB)

 18.2 MB 14.5 MB/s

Collecting pandas==2.2.2

Downloading pandas-2.2.2-cp39-cp39-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (13.1 MB)

 13.1 MB 74.8 MB/s

Collecting wheel==0.43.0

Downloading wheel-0.43.0-py3-none-any.whl (65 kB)

 65 kB 11.4 MB/s

Collecting scikit-learn==1.3.2

Downloading

scikit_learn-1.3.2-cp39-cp39-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (10.9 MB)

 10.9 MB 75.7 MB/s

Collecting matplotlib==3.8.4

Downloading matplotlib-3.8.4-cp39-cp39-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (11.6 MB)

 11.6 MB 70.6 MB/s

Collecting flask==3.0.3

Downloading flask-3.0.3-py3-none-any.whl (101 kB)

 101 kB 25.7 MB/s

Collecting python-dateutil>=2.8.2

Downloading python_dateutil-2.9.0.post0-py2.py3-none-any.whl (229 kB)

 229 kB 80.1 MB/s

Collecting pytz>=2020.1

Downloading pytz-2026.3.post1-py2.py3-none-any.whl (508 kB)

 508 kB 65.3 MB/s

Collecting tzdata>=2022.7

Downloading tzdata-2026.3-py2.py3-none-any.whl (348 kB)

 348 kB 73.0 MB/s

Collecting threadpoolctl>=2.0.0

Downloading threadpoolctl-3.6.0-py3-none-any.whl (18 kB)

Collecting joblib>=1.1.1

Downloading joblib-1.5.3-py3-none-any.whl (309 kB)

 309 kB 77.4 MB/s

Collecting scipy>=1.5.0

Downloading scipy-1.13.1-cp39-cp39-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (38.6 MB)

 38.6 MB 65.6 MB/s

Collecting fonttools>=4.22.0

Downloading

fonttools-4.60.2-cp39-cp39-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (4.8 MB)

 4.8 MB 72.5 MB/s

Collecting pillow>=8

Downloading pillow-11.3.0-cp39-cp39-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (6.6 MB)

 6.6 MB 68.7 MB/s

Collecting contourpy>=1.0.1

Downloading

contourpy-1.3.0-cp39-cp39-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (321 kB)

 321 kB 71.7 MB/s

Collecting kiwisolver>=1.3.1

Downloading

kiwisolver-1.4.7-cp39-cp39-manylinux_2_12_x86_64.manylinux2010_x86_64.whl (1.6 MB)

 1.6 MB 64.3 MB/s

Collecting packaging>=20.0

Using cached packaging-26.3-py3-none-any.whl (129 kB)

Collecting cycler>=0.10

Downloading cycler-0.12.1-py3-none-any.whl (8.3 kB)

Collecting pyparsing>=2.3.1

Downloading pyparsing-3.3.2-py3-none-any.whl (122 kB)

 122 kB 72.9 MB/s

Collecting importlib-resources>=3.2.0

Downloading importlib_resources-6.5.2-py3-none-any.whl (37 kB)

Collecting importlib-metadata>=3.6.0

Using cached importlib_metadata-8.7.1-py3-none-any.whl (27 kB)

Collecting itsdangerous>=2.1.2

Using cached itsdangerous-2.2.0-py3-none-any.whl (16 kB)

Collecting click>=8.1.3

Using cached click-8.1.8-py3-none-any.whl (98 kB)

Collecting Jinja2>=3.1.2

Using cached jinja2-3.1.6-py3-none-any.whl (134 kB)

Collecting blinker>=1.6.2

Using cached blinker-1.9.0-py3-none-any.whl (8.5 kB)

Collecting Werkzeug>=3.0.0

Using cached werkzeug-3.1.8-py3-none-any.whl (226 kB)

Collecting zipp>=3.20

Using cached zipp-3.23.1-py3-none-any.whl (10 kB)

Collecting MarkupSafe>=2.0

Using cached

markupsafe-3.0.3-cp39-cp39-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (20 kB)

Collecting six>=1.5

Downloading six-1.17.0-py2.py3-none-any.whl (11 kB)

Installing collected packages: zipp, six, numpy, MarkupSafe, Werkzeug, tzdata, threadpoolctl, scipy, pytz, python-dateutil, pyparsing, pillow, packaging, kiwisolver, joblib, Jinja2, itsdangerous, importlib-resources, importlib-metadata, fonttools, cycler, contourpy, click, blinker, wheel, scikit-learn, pandas, matplotlib, flask

Successfully installed Jinja2-3.1.6 MarkupSafe-3.0.3 Werkzeug-3.1.8 blinker-1.9.0 click-8.1.8
contourpy-1.3.0 cycler-0.12.1 flask-3.0.3 fonttools-4.60.2 importlib-metadata-8.7.1
importlib-resources-6.5.2 itsdangerous-2.2.0 joblib-1.5.3 kiwisolver-1.4.7 matplotlib-3.8.4
numpy-1.26.4 packaging-26.3 pandas-2.2.2 pillow-11.3.0 pyparsing-3.3.2
python-dateutil-2.9.0.post0 pytz-2026.3.post1 scikit-learn-1.3.2 scipy-1.13.1 six-1.17.0
threadpoolctl-3.6.0 tzdata-2026.3 wheel-0.43.0 zipp-3.23.1
WARNING: You are using pip version 21.3.1; however, version 26.0.1 is available.
You should consider upgrading via the
'/home/ec2-user/ckd_insurance/awslocalmodelhosting/venv/bin/python3 -m pip install --upgrade
pip' command.
(venv) [ec2-user@ip-172-31-47-206 awslocalmodelhosting]\$

=====

Excellent  The installation completed successfully.

The key line is:

Successfully installed ... flask-3.0.3 ... scikit-learn-1.3.2 ... numpy-1.26.4 ...

So your **CKD Insurance Flask project dependencies are ready**.

The pip upgrade warning is **not an error**. We can leave it for now.

=====

Replace following

```
# For more information on configuration, see:
# * Official English Documentation: http://nginx.org/en/docs/
# * Official Russian Documentation: http://nginx.org/ru/docs/

user nginx;
worker_processes auto;
error_log /var/log/nginx/error.log notice;
pid /run/nginx.pid;

# Load dynamic modules. See /usr/share/doc/nginx/README.dynamic.
include /usr/share/nginx/modules/*.conf;

events {
    worker_connections 1024;
```

```

}

http {
    log_format main '$remote_addr - $remote_user [$time_local] "$request" '
        '$status $body_bytes_sent "$http_referer" '
        '"$http_user_agent" "$http_x_forwarded_for"';

    access_log /var/log/nginx/access.log main;

    sendfile        on;
    tcp_nopush      on;
    keepalive_timeout 65;
    types_hash_max_size 4096;

    include         /etc/nginx/mime.types;
    default_type     application/octet-stream;

    # Load modular configuration files from the /etc/nginx/conf.d directory.
    # See http://nginx.org/en/docs/nginx_core_module.html#include
    # for more information.
    include /etc/nginx/conf.d/*.conf;

    server {
        listen      80;
        listen      [::]:80;
        server_name _;
        root         /usr/share/nginx/html;

        # Load configuration files for the default server block.
        include /etc/nginx/default.d/*.conf;

        error_page 404 /404.html;
        location = /404.html {
        }

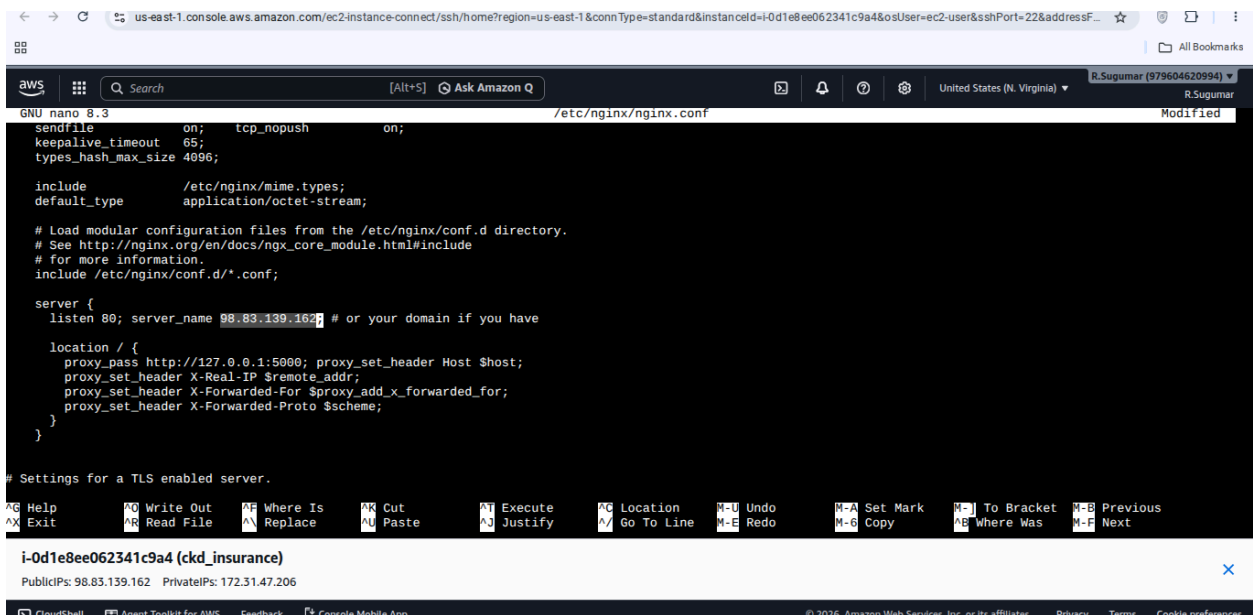
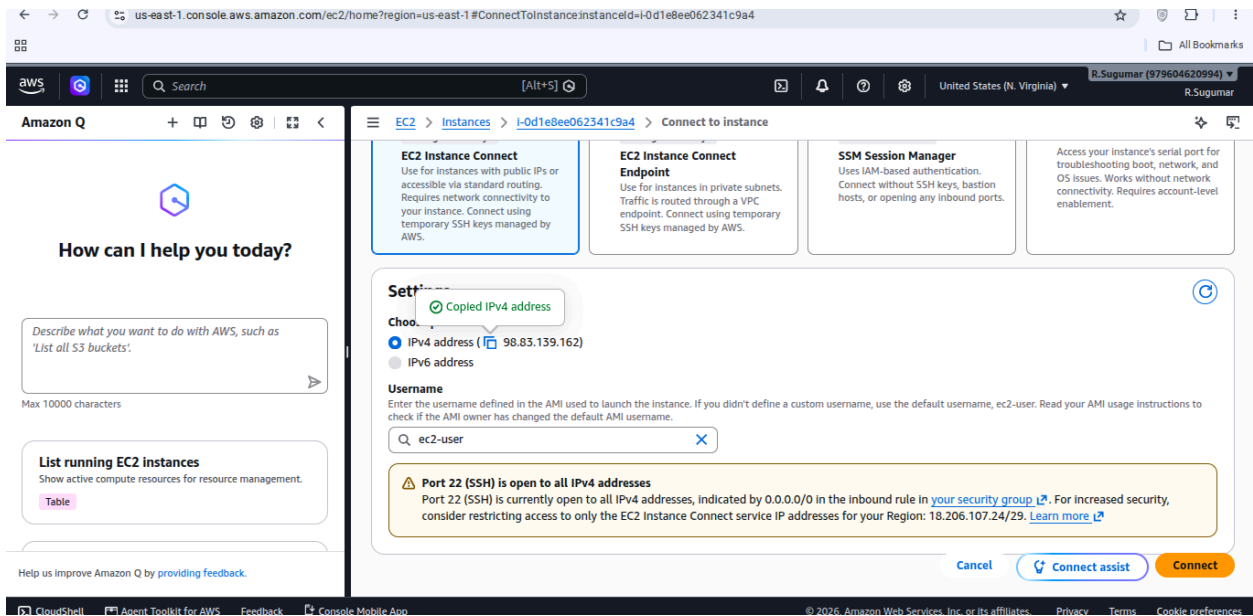
        # Settings for a TLS enabled server.
        #
        # server {
        #     listen      443 ssl;
        #     listen      [::]:443 ssl;
        #     http2        on;
        #     server_name _;
        #     root         /usr/share/nginx/html;
        #

```

```
#    ssl_certificate "/etc/pki/nginx/server.crt";
#    ssl_certificate_key "/etc/pki/nginx/private/server.key";
#    ssl_session_cache shared:SSL:1m;
#    ssl_session_timeout 10m;
#    ssl_ciphers PROFILE=SYSTEM;
#    ssl_prefer_server_ciphers on;
#
#    # Load configuration files for the default server block.
#    include /etc/nginx/default.d/*.conf;
#
#    error_page 404 /404.html;
#    location = /404.html
#    }
#
#    error_page 500 502 503 504 /50x.html;
#    location = /50x.html {
#    }
# }
}
```

=====

```
server {
    listen 80; server_name your-ec2-ip; # or your domain if you have
    location / {
        proxy_pass http://127.0.0.1:5000; proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```



====

Yes 👍 Your `nginx.conf` looks correctly configured for your Flask application.

The important part is already present:

```
server {  
    listen 80;
```

```
server_name 98.83.139.162;

location / {
    proxy_pass http://127.0.0.1:5000;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
}
```

So don't edit the file again.

====

Next step 1 — Test Nginx

Run:

```
sudo nginx -t
```

You want to see:

```
syntax is ok
test is successful
```

====

```
(venv) [ec2-user@ip-172-31-47-206 awslocalmodelhosting]$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
(venv) [ec2-user@ip-172-31-47-206 awslocalmodelhosting]$
```

=====

Next step 2 — Start Nginx

If the test succeeds, run:

```
sudo systemctl restart nginx
```

=====

Yes 👍 Since you are inside your project folder and the (`venv`) is active, this is the correct command to start your Flask app with **Gunicorn**:

```
gunicorn -w 3 -b 0.0.0.0:5000 app:app
```

What it means

- `gunicorn` → production web server
- `-w 3` → starts **3 worker processes**
- `-b 0.0.0.0:5000` → listens on port **5000**
- `app:app` → uses `app.py` and the Flask object named `app`

If successful, you'll see something similar to:

```
[INFO] Starting gunicorn
[INFO] Listening at: http://0.0.0.0:5000
[INFO] Using worker: sync
[INFO] Booting worker with pid: ...
```

===

```
(venv) [ec2-user@ip-172-31-47-206 awslocalmodelhosting]$ sudo systemctl restart nginx
(venv) [ec2-user@ip-172-31-47-206 awslocalmodelhosting]$ gunicorn -w 3 -b 0.0.0.0:5000
app:app
-bash: gunicorn: command not found
(venv) [ec2-user@ip-172-31-47-206 awslocalmodelhosting]$ gunicorn -w 3 -b 0.0.0.0:5000
app:app
-bash: gunicorn: command not found
(venv) [ec2-user@ip-172-31-47-206 awslocalmodelhosting]$
```

====

That's okay 👍 The problem is simple:

Gunicorn is not installed in this current virtual environment.

Earlier you installed Gunicorn in a different `venv`, but you later created a **new `venv` inside `awslocalmodelhosting`**. This new environment doesn't have Gunicorn.

Step 1 — Install Gunicorn

You are already in the correct folder and (venv) is active.

Run:

```
pip install gunicorn
```

Wait until you see:

```
Successfully installed gunicorn-...
```

===

```
(venv) [ec2-user@ip-172-31-47-206 awslocalmodelhosting]$ pip install gunicorn
Collecting gunicorn
  Using cached gunicorn-23.0.0-py3-none-any.whl (85 kB)
Requirement already satisfied: packaging in ./venv/lib/python3.9/site-packages (from gunicorn) (26.3)
Installing collected packages: gunicorn
Successfully installed gunicorn-23.0.0
WARNING: You are using pip version 21.3.1; however, version 26.0.1 is available.
You should consider upgrading via the
'/home/ec2-user/ckd_insurance/awslocalmodelhosting/venv/bin/python3 -m pip install --upgrade
pip' command.
(venv) [ec2-user@ip-172-31-47-206 awslocalmodelhosting]$
```

=====

Step 2 — Verify

Run:

```
gunicorn --version
```

You should get something like:

```
gunicorn (version 23.0.0)
```

=====

```
(venv) [ec2-user@ip-172-31-47-206 awslocalmodelhosting]$ gunicorn --version
gunicorn (version 23.0.0)
(venv) [ec2-user@ip-172-31-47-206 awslocalmodelhosting]$
```

=====

Step 3 — Start your Flask application

Then run:

```
gunicorn -w 3 -b 0.0.0.0:5000 app:app
```

===

```
(venv) [ec2-user@ip-172-31-47-206 awslocalmodelhosting]$ gunicorn -w 3 -b 0.0.0.0:5000
app:app
[2026-08-25 13:08:14 +0000] [42334] [INFO] Starting gunicorn 23.0.0
[2026-08-25 13:08:14 +0000] [42334] [INFO] Listening at: http://0.0.0.0:5000 (42334)
[2026-08-25 13:08:14 +0000] [42334] [INFO] Using worker: sync
[2026-08-25 13:08:15 +0000] [42335] [INFO] Booting worker with pid: 42335
[2026-08-25 13:08:15 +0000] [42336] [INFO] Booting worker with pid: 42336
[2026-08-25 13:08:15 +0000] [42337] [INFO] Booting worker with pid: 42337
/home/ec2-user/ckd_insurance/awslocalmodelhosting/venv/lib64/python3.9/site-packages/sklearn/base.py:348: InconsistentVersionWarning: Trying to unpickle estimator DecisionTreeClassifier
from version 1.3.0 when using version 1.3.2. This might lead to breaking code or invalid results.
Use at your own risk. For more info please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
/home/ec2-user/ckd_insurance/awslocalmodelhosting/venv/lib64/python3.9/site-packages/sklearn/base.py:348: InconsistentVersionWarning: Trying to unpickle estimator DecisionTreeClassifier
from version 1.3.0 when using version 1.3.2. This might lead to breaking code or invalid results.
Use at your own risk. For more info please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
/home/ec2-user/ckd_insurance/awslocalmodelhosting/venv/lib64/python3.9/site-packages/sklearn/base.py:348: InconsistentVersionWarning: Trying to unpickle estimator DecisionTreeClassifier
from version 1.3.0 when using version 1.3.2. This might lead to breaking code or invalid results.
Use at your own risk. For more info please refer to:
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
warnings.warn(
```

=====

Excellent 🎉 **Gunicorn is running successfully.**

Your output confirms:

Listening at: http://0.0.0.0:5000

and all **3 workers booted successfully**:

Booting worker with pid: 42335

Booting worker with pid: 42336

Booting worker with pid: 42337

About the warning

This is **not an error**:

InconsistentVersionWarning:

Trying to unpickle estimator DecisionTreeClassifier
from version 1.3.0 when using version 1.3.2

Your model was saved with **scikit-learn 1.3.0**, while your EC2 environment has **1.3.2**. The application has still started successfully.

Now test the website

Keep this terminal running. **Do not press Ctrl+C.**

Open a new browser tab and go to:

http://98.83.139.162

Your Nginx configuration should forward:

Browser → Port 80 → Nginx → Port 5000 → Gunicorn → Flask

====

Chronic Kidney Disease Prediction

Age:

Blood Pressure:

Specific Gravity:

Albumin:

Sugar:

🎉 **Success! Your CKD Insurance application is live on AWS EC2.**

The screenshot confirms the complete deployment is working:

- **EC2:** Running ✓
- **Gunicorn:** Running on port **5000** ✓
- **Nginx:** Forwarding port **80** to Gunicorn ✓
- **Public IP:** **98.83.139.162** ✓
- **Flask application:** Loading successfully ✓
- **CKD Prediction form:** Displayed correctly ✓

Your deployment flow is now:

```
graph TD
    Internet --> IP[98.83.139.162:80]
    IP --> Nginx
    Nginx --> Gunicorn[Gunicorn :5000]
    Gunicorn --> Flask[Flask app.py]
    Flask --> Model[CKD Prediction Model]
```

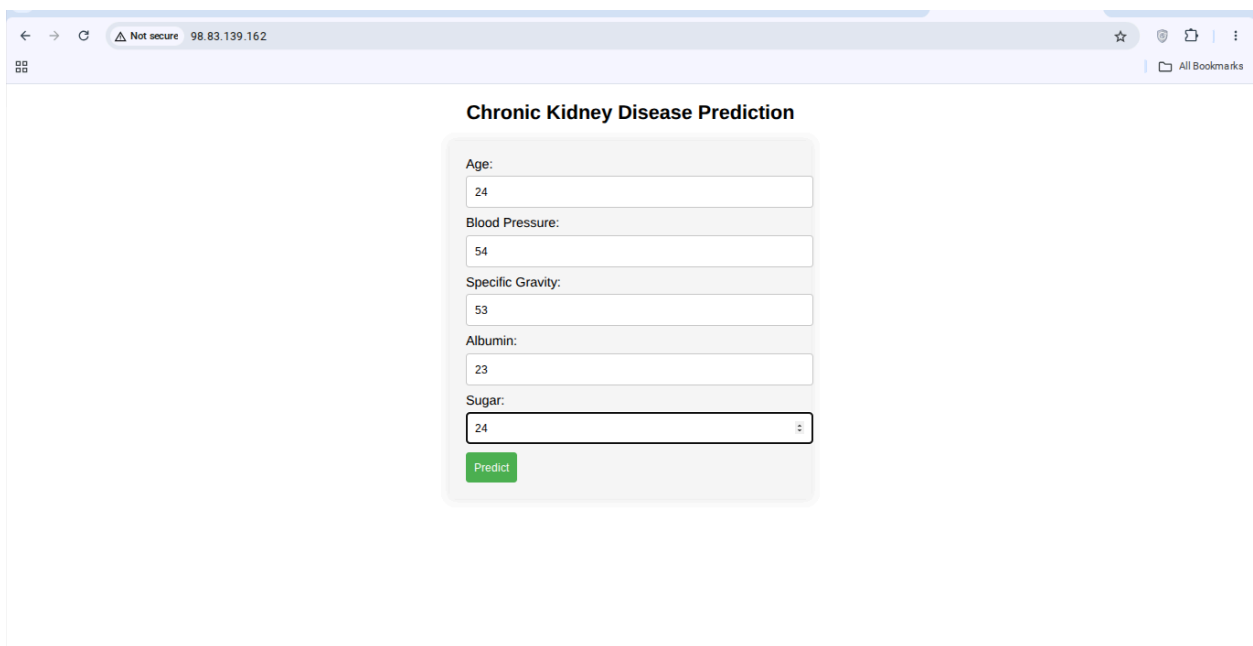
One important next step

Right now Gunicorn is running **only in your terminal**. If you close the terminal or stop the process, the website will stop.

The next step should be to create a **systemd service for Gunicorn**, so it automatically runs in the background and can restart if the server reboots.

We should do that **before considering the deployment complete**.

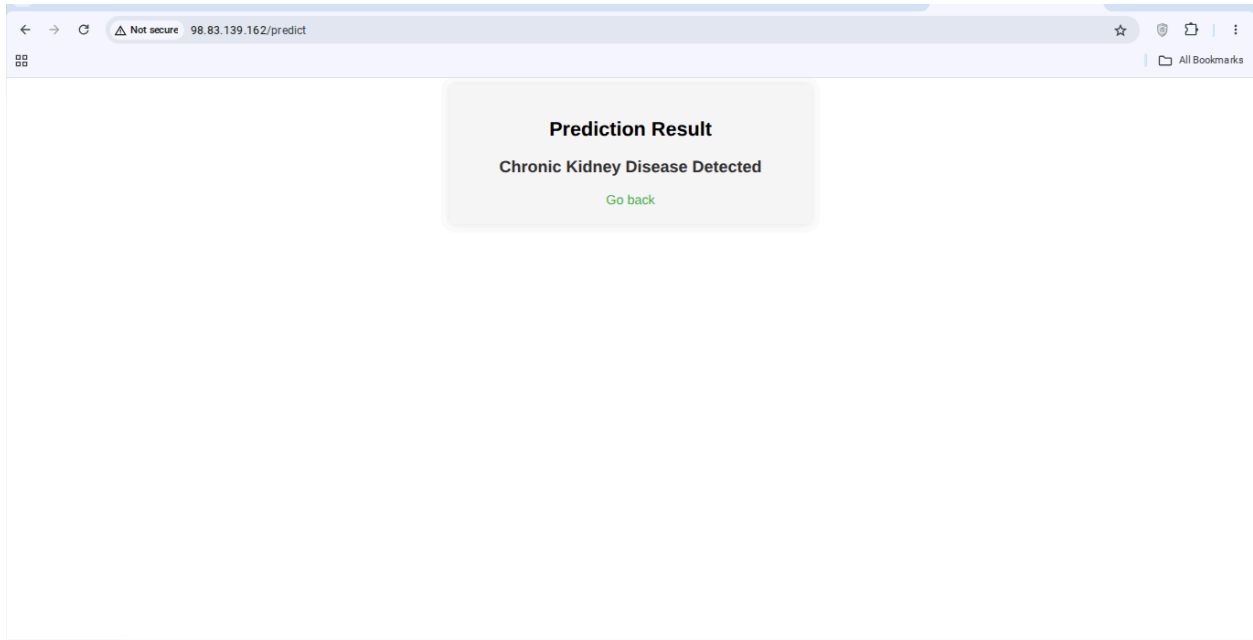
====



The screenshot shows a web browser window with the address bar displaying 'Not secure 98.83.139.162'. The page title is 'Chronic Kidney Disease Prediction'. The form contains the following fields and values:

Field	Value
Age:	24
Blood Pressure:	54
Specific Gravity:	53
Albumin:	23
Sugar:	24

Below the input fields is a green button labeled 'Predict'.



===

Yes — the prediction is working successfully. 

Your browser reached:

98.83.139.162/predict

and the application returned:

Prediction Result
Chronic Kidney Disease Detected

So the complete flow is working:

User Input



Flask



Trained CKD Model



Prediction



Result Page

One thing to remember

The **"Not secure"** message is expected because you're using:

`http://98.83.139.162`

rather than HTTPS. It does **not** mean your Flask application is broken.

Next step

Your application is currently running through the terminal with:

```
gunicorn -w 3 -b 0.0.0.0:5000 app:app
```

The next important step is to make Gunicorn a **background system service**, so your application continues running even after you close the EC2 terminal or restart the EC2 instance.

====

Great. Your application is working. 

The **next step is to make Gunicorn run automatically in the background**. Currently, if you close the terminal, Gunicorn will stop.

Step 1 — Stop the current Gunicorn

Go to the terminal where Gunicorn is running and press:

Ctrl + C

You should return to:

```
(venv) [ec2-user@ip-172-31-47-206 awslocalmodelhosting]$
```

Step 2 — Create a Gunicorn service

Run:

```
sudo nano /etc/systemd/system/ckd_insurance.service
```

Paste **exactly this**:

```
[Unit]
Description=CKD Insurance Flask Application
After=network.target
```

```
[Service]
User=ec2-user
Group=ec2-user
WorkingDirectory=/home/ec2-user/ckd_insurance/awslocalmodelhosting
ExecStart=/home/ec2-user/ckd_insurance/awslocalmodelhosting/venv/bin/gunicorn -w 3 -b
127.0.0.1:5000 app:app
Restart=always

[Install]
WantedBy=multi-user.target
```

Step 3 — Save

In nano:

```
Ctrl + O
Enter
Ctrl + X
```

Step 4 — Reload systemd

```
sudo systemctl daemon-reload
```

Step 5 — Start your application

```
sudo systemctl start ckd_insurance
```

Step 6 — Enable it automatically

```
sudo systemctl enable ckd_insurance
```

Step 7 — Check status

```
sudo systemctl status ckd_insurance
```

You want to see:

Active: active (running)

Your final architecture will then be:

```
Internet
↓
EC2 Public IP :80
↓
Nginx
↓
```


Gunicorn :5000



Flask app.py



CKD Model

Do Steps 1–7, and send me the `systemctl status ckd_insurance` output.

=====

Excellent 👍 Now your Flask application should run **automatically in the background**.

Step 1 — Verify Gunicorn service

Run:

```
sudo systemctl status ckd_insurance
```

You should see:

Active: active (running)

Step 2 — Test from the EC2 server

Run:

```
curl http://127.0.0.1:5000
```

You should receive HTML from your Flask application.

Step 3 — Test through Nginx

Run:

```
curl http://98.83.139.162
```

You should again receive your CKD application HTML.

Step 4 — Test in your browser

Open:

```
http://98.83.139.162
```

Then enter the CKD values and click **Predict**.

Step 5 — Confirm it survives terminal closure

You can now safely close the EC2 Instance Connect terminal and reopen it later. Unicorn should continue running because it is managed by:

systemd → ckd_insurance.service

Important AWS point

Your current public IP:

98.83.139.162

is an **EC2 public IPv4 address**. If you **stop and start** the instance, AWS can assign a different public IP.

For a permanent website address, the next deployment step is to allocate and associate an **Elastic IP**.

So after verifying the service, the recommended next step is:

Elastic IP → Security hardening → HTTPS/SSL → final deployment.

====

Perfect 👍 Your application is now running as a **systemd service**. The next step is to make the IP address permanent using an **Elastic IP**.

Step 1 — Open Elastic IPs

In AWS Console:

EC2 → Network & Security → Elastic IPs

Then click:

Allocate Elastic IP address

Step 2 — Allocate

Leave the default settings and click:

Allocate

AWS will create an Elastic IP.

Step 3 — Associate it with your instance

Select the new Elastic IP → **Actions** → **Associate Elastic IP address**

For **Resource type**, choose:

Instance

For **Instance**, select:

ckd_insurance

Then click:

Associate

Step 4 — Check the new IP

Go to:

EC2 → **Instances** → **ckd_insurance**

Look at **Public IPv4 address**.

It should now show your new Elastic IP.

Step 5 — Test your application

Open:

http://YOUR-ELASTIC-IP

Your **Chronic Kidney Disease Prediction** page should appear.

 **Important:** Don't use the old **98.83.139.162** address after associating the Elastic IP. Use the new Elastic IP shown in the EC2 instance details.

After this, the next step will be **HTTPS/SSL**, so your site can use **https://** instead of showing **Not secure**.

=====

← → ↺

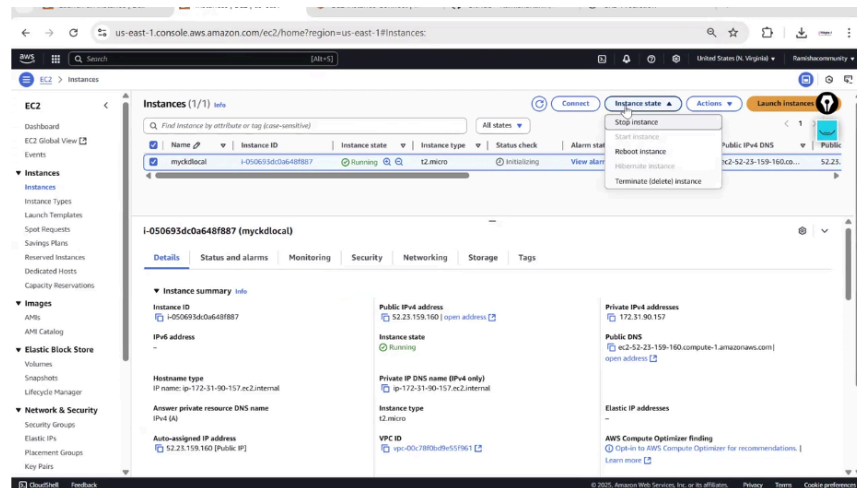
courses.hopelearning.net/myaccount/#/course/81394/lesson/1649274?lesson=1649274&lesson_type=material§ion=374298&subject=222674

☆ 🔒 📄

All Bookmarks

24.Deploying Local model on EC2 Instance

Complete & Continue



After finished project, delete it

=====